

SIMATS ENGINEERING

CO5 - ASSESSMENT TOOL 2

Mock Interview

COURSE NAME: Deep Learning

COURSE CODE: MLA0408

COURSE OUTCOME COVERED

CO5: Students will be able to evaluate advanced machine learning concepts such as reinforcement learning, Monte Carlo methods, sampling techniques, policy iteration, value iteration, and temporal difference learning. They will understand computational learning theory concepts including VC dimension, sample complexity, and mistake-bound analysis. Students will be capable of selecting and optimizing advanced learning strategies for intelligent systems and complex applications. (BL-6)

Assessment Tool Weightage: 20%

QUESTIONS:

Q1. Autoencoder Design

- What is an autoencoder and what are its main components in a neural network architecture?
- How would you design an autoencoder for reducing the dimensionality of a highdimensional dataset?
- What factors would you consider when deciding the size of the encoder and decoder layers?
- Why is the bottleneck layer important in an autoencoder design?
- How would you choose an appropriate activation function for the encoder and decoder of an autoencoder?

Q2. Stochastic Encoders and Decoders

- What is a stochastic encoder, and how does it differ from a deterministic encoder?
- Why is randomness introduced into the encoding process of a stochastic autoencoder?
- How does a stochastic decoder reconstruct data from a probabilistic latent representation?

- What are the advantages of using stochastic encoders and decoders in generative models?
- How does a stochastic encoder represent uncertainty in the input data?

Q3. Deep Belief Network vs Deep Boltzmann Machine

- What is the main architectural difference between a Deep Belief Network (DBN) and a Deep Boltzmann Machine (DBM)?
- How are connections between hidden layers organized differently in a DBN and a DBM?
- Why is a DBN considered a hybrid model containing both directed and undirected connections?
- How does a DBM differ from a DBN in terms of the direction of connections between layers?
- How does the training procedure of a DBN differ from that of a DBM?

Q4. Generative Adversarial Networks

- What is a Generative Adversarial Network, and what are its two main components?
- How does the generator in a GAN create new data samples?
- What is the primary role of the discriminator in a GAN?
- How do the generator and discriminator compete with each other during GAN training?
- Why is balancing the generator and discriminator important for stable GAN training?

Q5. Integrated Deep Learning Solution

- What do you understand by an integrated deep learning solution, and what are its major components?
- How would you combine different deep learning models to solve a complex real-world problem?
- How would you decide whether to use an autoencoder, DBN, DBM, or GAN in an integrated solution?
- What factors should be considered when designing an end-to-end deep learning pipeline?
- How can feature extraction and representation learning be integrated into a deep learning solution?

Code of Conduct:

I **K. Hema sai Siddartha (192325036)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Hema sai Siddartha

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 Exemplary	Level 3 Proficient	Level 2 Developing	Level 1 Beginning
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problemsolving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides wellstructured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism

ANSWERS

Q1. Autoencoder Design

Q: What is an autoencoder and what are its main components in a neural network architecture?

A: An autoencoder is an unsupervised neural network trained to reconstruct its own input by first compressing it into a lower-dimensional representation and then rebuilding it. Its main components are: (1) the Encoder, which maps the input x to a compressed latent representation $z = f(x)$; (2) the Latent space / bottleneck, the compressed code that holds the most important features of the input; and (3) the Decoder, which reconstructs the input from the latent code as $x' = g(z)$. The network is trained by minimizing a reconstruction loss (e.g., mean squared error or binary cross-entropy) between the original input x and the reconstructed output x' .

Q: How would you design an autoencoder for reducing the dimensionality of a highdimensional dataset?

A: I would first analyze the input dimensionality and data type to decide the network depth (dense layers for tabular data, convolutional layers for images). I would design a symmetric architecture where the encoder progressively reduces the number of neurons per layer (e.g., $784 \rightarrow 256 \rightarrow 64 \rightarrow 32$) down to the desired bottleneck size, and the decoder mirrors this in reverse ($32 \rightarrow 64 \rightarrow 256 \rightarrow 784$). I would normalize the input data, choose a reconstruction loss appropriate to the data range, use regularization (dropout/L2) to prevent overfitting, and validate the reduced dimensionality by checking reconstruction error and visualizing the latent space (e.g., using t-SNE) to confirm that meaningful structure is preserved.

Q: What factors would you consider when deciding the size of the encoder and decoder layers?

A: Key factors include: the intrinsic dimensionality and complexity of the data (more complex data needs more capacity), the amount of available training data (larger layers risk overfitting on small datasets), the desired compression ratio and downstream use of the latent code, computational/memory constraints, and empirical validation through reconstruction loss on a held-out set. I would also consider a gradual, funnel-shaped reduction rather than an abrupt drop in layer size, since sudden compression can cause loss of important information.

Q: Why is the bottleneck layer important in an autoencoder design?

A: The bottleneck layer forces the network to learn a compressed, information-dense representation of the input rather than simply copying it. This constraint compels the model to capture only the most salient and generalizable features, which is what enables the autoencoder to perform meaningful dimensionality reduction, denoising, and feature learning. Without a bottleneck (or an equivalent constraint such as sparsity), the network could trivially learn the identity function and would not learn any useful structure.

Q: How would you choose an appropriate activation function for the encoder and decoder of an autoencoder?

A: For hidden layers in both encoder and decoder, I would generally use ReLU (or a variant like Leaky ReLU) because it mitigates vanishing gradients and trains efficiently. For the output layer of the decoder, the choice depends on the input data range: a sigmoid activation is appropriate when inputs are normalized to $[0,1]$ (e.g., pixel intensities), while a linear (identity) activation is

used when inputs are continuous and unbounded. The guiding principle is that the decoder's final activation should match the statistical range and distribution of the original input data so the reconstruction loss is well defined.

Q2. Stochastic Encoders and Decoders

Q: What is a stochastic encoder, and how does it differ from a deterministic encoder?

A: A stochastic encoder maps an input x to a probability distribution over the latent space (typically parameters of a Gaussian, i.e., a mean and variance) rather than to a single fixed point. A latent vector z is then sampled from this distribution, $z \sim q(z|x)$. In contrast, a deterministic encoder produces one fixed latent vector $z = f(x)$ for a given input, with no sampling involved. The stochastic approach, used in models like Variational Autoencoders (VAEs), introduces controlled randomness that shapes a smooth, continuous latent space.

Q: Why is randomness introduced into the encoding process of a stochastic autoencoder?

A: Randomness is introduced so the model learns a smooth, continuous, and well-structured latent space instead of memorizing discrete points for each training example. By sampling nearby points during training, the decoder is forced to produce plausible outputs for a neighborhood around each encoded point, not just the exact training instance. This enables meaningful interpolation between data points, better generalization, and the ability to generate novel, realistic samples by sampling from the latent distribution.

Q: How does a stochastic decoder reconstruct data from a probabilistic latent representation?

A: A stochastic decoder takes a latent sample z (drawn from the encoder's distribution, or from the prior when generating new data) and outputs the parameters of a probability distribution over the data space, such as the mean of a Gaussian for continuous data or probabilities for a Bernoulli distribution for binary data. The final reconstruction is either the expected value of this distribution or a sample drawn from it, which allows the model to express uncertainty in the reconstructed output rather than producing a single deterministic value.

Q: What are the advantages of using stochastic encoders and decoders in generative models?

A: Advantages include: the ability to generate new, realistic data samples by sampling from the latent prior; a smoother and more continuous latent space that supports meaningful interpolation; improved regularization, since the model cannot simply memorize exact latent codes; better handling of ambiguous or noisy inputs by representing uncertainty explicitly; and a principled probabilistic framework (as in VAEs) that connects the model to variational inference, enabling well-calibrated generative behavior.

Q: How does a stochastic encoder represent uncertainty in the input data?

A: Instead of collapsing an input to a single point, the stochastic encoder outputs a mean and variance (or standard deviation) for each latent dimension. The variance term quantifies how confident the model is about the encoding: a small variance indicates high confidence in a specific latent value, while a larger variance reflects greater uncertainty, meaning several nearby latent points are almost equally plausible representations of the input. This variance is learned jointly with the mean during training via the reparameterization trick.

Q3. Deep Belief Network vs Deep Boltzmann Machine

Q: What is the main architectural difference between a Deep Belief Network (DBN) and a Deep Boltzmann Machine (DBM)?

A: A DBN is a hybrid graphical model composed of several stacked Restricted Boltzmann Machines (RBMs) where only the top two layers have undirected connections, while all lower layers have directed, top-down connections (like a belief network). A DBM, on the other hand, is a fully undirected graphical model — every pair of connected layers has symmetric, undirected connections throughout the entire network, making it a single, uniform Boltzmann machine rather than a hybrid stack.

Q: How are connections between hidden layers organized differently in a DBN and a DBM?

A: In a DBN, only the connections between the top two hidden layers are undirected; all other inter-layer connections are directed downward from higher to lower layers. In a DBM, connections between every pair of adjacent hidden layers (and between the visible and first hidden layer) are undirected throughout the network, allowing information to flow bidirectionally between all layers during inference.

Q: Why is a DBN considered a hybrid model containing both directed and undirected connections?

A: A DBN is built by stacking RBMs and then treating the top two layers as an undirected associative memory, while the connections in the lower layers are directed, since they are derived from the generative (top-down) structure used to reconstruct visible units from the layer above during greedy layer-wise pretraining. This combination of an undirected top RBM with directed lower layers is why DBNs are described as hybrid directed-undirected models.

Q: How does a DBM differ from a DBN in terms of the direction of connections between layers?

A: In a DBM, all connections are undirected and symmetric between every adjacent pair of layers, so information can propagate both bottom-up and top-down simultaneously during inference. In a DBN, only the top two layers are undirected; the connections in all lower layers are directed from top to bottom, meaning inference in a DBN is primarily a one-directional, layer-by-layer generative process rather than the fully bidirectional process used in a DBM.

Q: How does the training procedure of a DBN differ from that of a DBM?

A: A DBN is trained using greedy, layer-wise pretraining: each RBM is trained independently, bottom to top, treating the hidden layer of one RBM as the visible layer for the next, and this can optionally be followed by fine-tuning with backpropagation (e.g., wake-sleep algorithm). A DBM, being fully undirected, must be trained jointly considering all layers simultaneously; it typically uses a combination of greedy layer-wise pretraining for initialization followed by a joint training procedure involving mean-field variational inference for the approximate posterior and persistent contrastive divergence (or stochastic approximation) for the model's negative phase, making DBM training more computationally intensive.

Q4. Generative Adversarial Networks

Q: What is a Generative Adversarial Network, and what are its two main components?

A: A Generative Adversarial Network (GAN) is a generative deep learning framework in which two neural networks are trained simultaneously in an adversarial (competitive) setting. Its two main components are the Generator, which learns to produce synthetic data samples that resemble the real data distribution, and the Discriminator, which learns to distinguish between real samples (from the training data) and fake samples (produced by the generator). Both networks improve together through this competition.

Q: How does the generator in a GAN create new data samples?

A: The generator takes a random noise vector z sampled from a simple prior distribution (such as a standard normal or uniform distribution) as input and passes it through a series of (typically transposed convolutional or dense) layers that progressively transform this noise into a structured output, such as an image, that resembles samples from the real data distribution. During training, the generator's weights are updated to make these outputs increasingly convincing to the discriminator.

Q: What is the primary role of the discriminator in a GAN?

A: The discriminator acts as a binary classifier whose primary role is to distinguish between real data samples drawn from the training set and fake samples produced by the generator, outputting a probability that a given sample is real. This feedback signal is what drives the generator to improve, since the generator's objective is explicitly to fool the discriminator into misclassifying its fake outputs as real.

Q: How do the generator and discriminator compete with each other during GAN training?

A: Training is framed as a two-player minimax game: the discriminator is trained to maximize its accuracy in correctly labeling real versus fake samples, while the generator is trained to minimize the discriminator's ability to tell its outputs apart from real data. The two networks are updated alternately — the discriminator's weights are adjusted to better catch fakes, and then the

generator's weights are adjusted using the discriminator's feedback to produce more realistic samples — so improvements in one network push the other to improve as well.

Q: Why is balancing the generator and discriminator important for stable GAN training?

A: If the discriminator becomes far stronger than the generator too quickly, it can perfectly distinguish real from fake samples, causing the generator's gradients to vanish and training to stall (the generator receives no useful learning signal). Conversely, if the generator overpowers the discriminator, the discriminator's feedback becomes uninformative and can lead to mode collapse, where the generator produces a limited variety of outputs. Careful balancing — through learning rate tuning, architectural choices, and techniques such as label smoothing or gradient penalties — keeps both networks improving together, which is essential for stable convergence and high-quality generated samples.

Q5. Integrated Deep Learning Solution

Q: What do you understand by an integrated deep learning solution, and what are its major components?

A: An integrated deep learning solution combines multiple deep learning models and techniques into a unified pipeline to solve a complex, multi-faceted real-world problem that a single model cannot fully address. Its major components typically include: a data preprocessing and representation-learning stage (often using autoencoders for dimensionality reduction or denoising), a feature-learning/generative stage (using models such as DBNs, DBMs, or GANs for pretraining, feature extraction, or data augmentation), a task-specific supervised model (e.g., a classifier or regressor built on the learned representations), and an evaluation/feedback loop that monitors performance and refines the pipeline.

Q: How would you combine different deep learning models to solve a complex real-world problem?

A: I would first decompose the problem into sub-tasks — such as data cleaning, dimensionality reduction, feature learning, and final prediction/generation — and match each sub-task to the model best suited for it. For example, I might use an autoencoder for unsupervised dimensionality reduction and denoising, a DBN for greedy unsupervised pretraining of feature detectors, and a GAN for augmenting scarce training data or generating synthetic samples. These components would be connected in a pipeline where the output (learned representation) of one model becomes the input to the next, and the entire pipeline would be validated end-to-end, with fine-tuning applied to jointly optimize performance.

Q: How would you decide whether to use an autoencoder, DBN, DBM, or GAN in an integrated solution?

A: The choice depends on the specific objective: I would use an autoencoder when the goal is dimensionality reduction, denoising, or anomaly detection through reconstruction error; a DBN when I need fast, greedy layer-wise unsupervised pretraining to initialize a deep network before

fine-tuning, especially with limited labeled data; a DBM when I need a richer, fully undirected generative model that can capture more complex bidirectional dependencies between features, at the cost of higher training complexity; and a GAN when the primary requirement is generating new, realistic synthetic samples or performing data augmentation. In practice, the decision is guided by the nature of the task (reconstruction vs. generation vs. pretraining), data availability, and computational budget.

Q: What factors should be considered when designing an end-to-end deep learning pipeline?

A: Important factors include: the quality, volume, and nature of available data; appropriate preprocessing and normalization steps; choice of architecture(s) suited to the sub-tasks involved; computational resources and training time constraints; strategies to prevent overfitting (regularization, dropout, data augmentation); a clear evaluation metric aligned with the business/real-world objective; scalability and deployment considerations; and a feedback mechanism to monitor model drift and retrain the pipeline as new data becomes available.

Q: How can feature extraction and representation learning be integrated into a deep learning solution?

A: Feature extraction and representation learning can be integrated by using unsupervised models — such as autoencoders, DBNs, or DBMs — as a pretraining stage that learns compact, informative representations of the raw data without requiring labels. These learned representations (the latent codes or hidden-layer activations) can then be fed into a supervised model for the final task (classification, regression, etc.), which often improves performance, especially in low-labeled-data settings. Additionally, GANs can be integrated to augment the training set with synthetic examples, further enriching the representations the downstream model learns from.